



faça uma pesquisa aprofundada sobre responsividade de sites e eaplicação em código

Vou criar o arquivo de exemplo usando texto direto:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exemplos Práticos de Responsividade Web</title>
  <style>
    /* ===== RESET BÁSICO ===== */
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: Arial, sans-serif;
      line-height: 1.6;
      color: #333;
    }

    /* ===== EXEMPLO 1: HEADER RESPONSIVO COM FLEXBOX ===== */
    .header {
      background: #4CAF50;
      color: white;
      padding: 1em 0;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }

    .header h1 {
      font-size: 1.5rem;
    }

    .nav-menu {
      display: flex;
      gap: 20px;
    }

    .nav-menu a {
      color: white;
      text-decoration: none;
    }
```

```

        padding: 8px 16px;
    }

    /* Media Query para mobile */
    @media (max-width: 768px) {
        .header {
            flex-direction: column;
            text-align: center;
        }

        .nav-menu {
            flex-direction: column;
            gap: 10px;
            margin-top: 15px;
        }

        .header h1 {
            font-size: 1.2rem;
        }
    }

    /* ===== EXEMPLO 2: GRID LAYOUT RESPONSIVO ===== */
    .container {
        display: grid;
        grid-template-columns: 1fr 3fr;
        grid-template-rows: auto 1fr auto;
        height: 100vh;
    }

    .sidebar {
        background: #f4f4f4;
        padding: 15px;
    }

    .content {
        padding: 15px;
        display: flex;
        flex-direction: column;
    }

    .footer {
        background: #333;
        color: white;
        text-align: center;
        padding: 1em 0;
        grid-column: 1 / -1;
    }

    /* Grid responsivo com media query */
    @media (max-width: 768px) {
        .container {
            grid-template-columns: 1fr;
            height: auto;
        }
    }

```

```

/* ===== EXEMPLO 3: CARDS RESPONSIVOS AUTO-FLEXÍVEIS ===== */
.cards-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
  margin: 20px 0;
}

.card {
  background: #e2e2e2;
  padding: 20px;
  border-radius: 8px;
}

.card h3 {
  margin-bottom: 10px;
  font-size: 1.2rem;
}

/* ===== EXEMPLO 4: FONTES RESPONSIVAS COM CLAMP ===== */
.responsive-text {
  font-size: clamp(1rem, 2vw, 2rem);
  margin: 20px 0;
}

h2 {
  font-size: clamp(1.5rem, 3vw, 2.5rem);
  margin: 20px 0;
}

/* ===== EXEMPLO 5: UNIDADES RELATIVAS (rem, em, %) ===== */
.relative-units {
  font-size: 1rem;
  margin: 1.5em 0;
  padding: 1rem;
}

/* ===== EXEMPLO 6: IMAGENS RESPONSIVAS ===== */
.image-container {
  width: 100%;
  margin: 20px 0;
}

.image-container img {
  max-width: 100%;
  height: auto;
  border-radius: 8px;
}

/* ===== EXEMPLO 7: VÍDEO RESPONSIVO ===== */
.video-container {
  position: relative;
  width: 100%;
  padding-bottom: 56.25%;
  height: 0;
  margin: 20px 0;
}

```

```

    }

    .video-container video {
        position: absolute;
        top: 0;
        left: 0;
        width: 100%;
        height: 100%;
    }

    /* ===== EXEMPLO 8: TABLET BREAKPOINT ===== */
    @media (min-width: 769px) and (max-width: 1024px) {
        .cards-grid {
            grid-template-columns: repeat(2, 1fr);
        }
    }

    /* ===== EXEMPLO 9: DESKTOP GRANDE BREAKPOINT ===== */
    @media (min-width: 1400px) {
        .container {
            grid-template-columns: 250px 1fr;
        }

        h2 {
            font-size: 2.5rem;
        }
    }

    /* ===== EXEMPLO 10: ORIENTAÇÃO DE TELAS ===== */
    @media (orientation: portrait) {
        .header h1 {
            font-size: 1rem;
        }
    }

    @media (orientation: landscape) {
        .header h1 {
            font-size: 1.5rem;
        }
    }
</style>
</head>
<body>
    <!-- EXEMPLO 1: Header Responsivo -->
    <header class="header">
        <h1>Site Responsivo</h1>
        <nav class="nav-menu">
            <a href="#">Home</a>
            <a href="#">Sobre</a>
            <a href="#">Serviços</a>
            <a href="#">Contato</a>
        </nav>
    </header>

    <div class="container">
        <!-- EXEMPLO 2: Sidebar e Conteúdo -->

```

```

<aside class="sidebar">
  <h3>Menu</h3>
  <ul>
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li>
  </ul>
</aside>

<main class="content">
  <!-- EXEMPLO 4: Textos Responsivos -->
  <h2>Tutorial de Responsividade Web</h2>
  <p class="responsive-text">
    Este texto usa clamp() para se adaptar automaticamente a qualquer tamanho.
  </p>

  <!-- EXEMPLO 3: Cards Grid Auto-Flexível -->
  <div class="cards-grid">
    <div class="card">
      <h3>Flexbox</h3>
      <p>LAYOUT unidimensional para alinhamento eficiente de elementos</p>
    </div>
    <div class="card">
      <h3>CSS Grid</h3>
      <p>LAYOUT bidimensional para layouts complexos e estruturados.</p>
    </div>
    <div class="card">
      <h3>Media Queries</h3>
      <p>Estilos condicionais baseados em características do dispositivo</p>
    </div>
    <div class="card">
      <h3>Unidades Relativas</h3>
      <p>rem, em, %, vw, vh para escalabilidade automática.</p>
    </div>
  </div>

  <!-- EXEMPLO 5: Unidades Relativas -->
  <div class="relative-units">
    <h3>Unidades Relativas em Prática</h3>
    <p>font-size: 1rem (16px base)</p>
    <p>margin: 1.5em (relativo ao font-size)</p>
    <p>padding: 1rem (escalável)</p>
  </div>

  <!-- EXEMPLO 6: Imagem Responsiva -->
  <div class="image-container">
    
  </div>

```

```

        <!-- EXEMPLO 7: Vídeo Responsivo -->
        <div class="video-container">
            <video controls>
                <source src="movie.mp4" type="video/mp4">
                Seu navegador não suportará vídeo.
            </video>
        </div>
    </main>
</div>

    <footer class="footer">
        <p>&copy; 2024 - Exemplos de Responsividade Web</p>
    </footer>
</body>
</html>

```

🔍 Pesquisa Aprofundada sobre Responsividade de Sites

1. Conceitos Fundamentais

A **responsividade web** é a capacidade de um site se adaptar automaticamente ao tamanho da tela do dispositivo, oferecendo experiência consistente em smartphone, tablet e desktop. ^[1] ^[2]

Três elementos essenciais do design responsivo: ^[3]

Elemento	Descrição
Layout fluido	Medidas flexíveis (porcentagens, unidades relativas)
Media queries	Ajustes de design baseados em características do dispositivo
Imagens flexíveis	Conteúdo de mídia redimensionável

2. Técnicas Modernas de Implementação

CSS Grid (Bidimensional)

Ideal para layouts complexos e estruturados: ^[4] ^[5]

```

.container {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 20px;
}

/* Responsividade: muda para coluna única em mobile */
@media (max-width: 768px) {
    .container {
        grid-template-columns: 1fr;
    }
}

```

```
}  
}
```

Flexbox (Unidimensional)

Excelente para alinhamento eficiente em uma direção:^[6] ^[4]

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
  justify-content: space-between;  
}  
  
.item {  
  flex: 1 1 200px; /* cresce, diminui, base 200px */  
}
```

3. Melhores Práticas em 2024-2025^[4]

Prática	Descrição
Mobile-First	Desenvolver primeiro versão móvel, expandir para telas maiores (aumento de 30% na velocidade) ^[4]
Breakpoints estratégicos	768px (tablet), 1024px (desktop), 1400px (grande desktop) ^[4]
Unidades relativas	Usar rem, em, %, vw, vh ao invés de pixels ^[7] ^[8]
Imagens responsivas	Usar srcset e sizes para servir tamanhos diferentes ^[4] ^[9]
Fontes adaptáveis	clamp() para texto que escala automaticamente ^[4] ^[10]

4. Unidades Relativas em CSS^[7] ^[8]

```
/* EXEMPLOS PRÁTICOS */  
font-size: 1rem;      /* 16px base (raiz) */  
font-size: 2em;       /* 2x font-size do pai */  
margin: 1.5em 0;      /* relativo ao font-size */  
width: 100%;          /* 100% do pai */  
font-size: 2vw;       /* 2% da largura da viewport */  
height: 50vh;         /* 50% da altura da viewport */
```

Diferença rem vs em:^[8]

- rem: relativo ao font-size do **html** (raiz)
- em: relativo ao font-size do **elemento pai**

5. Fontes Responsivas com `clamp()` ^[10]

```
/* Mínimo: 1rem, Preferencial: 2vw, Máximo: 2rem */
font-size: clamp(1rem, 2vw, 2rem);

/* Título: mínimo 1.5rem, preferred 3vw, máximo 2.5rem */
h2 {
  font-size: clamp(1.5rem, 3vw, 2.5rem);
}
```

Funciona assim:

- Em telas pequenas: usa o **mínimo** (1rem)
- Em telas médias: usa o **valor preferencial** (2vw)
- Em telas grandes: usa o **máximo** (2rem)

6. Imagens Responsivas com `srcset` e `sizes` ^{[9] [11]}

```

```

O navegador automaticamente escolhe a melhor imagem baseada no dispositivo. ^[9]

7. Media Queries - Breakpoints Comuns ^{[1] [4]}

```
/* MOBILE (até 768px) */
@media (max-width: 768px) {
  .container {
    grid-template-columns: 1fr;
  }
}

/* TABLET (769px - 1024px) */
@media (min-width: 769px) and (max-width: 1024px) {
  .cards-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* DESKTOP GRANDE (1400px+) */
@media (min-width: 1400px) {
  .container {
```



```
        grid-template-columns: 250px 1fr;
    }
}

/* ORIENTAÇÃO (portrait/landscape) */
@media (orientation: portrait) {
    h1 { font-size: 1rem; }
}

@media (orientation: landscape) {
    h1 { font-size: 1.5rem; }
}
```

8. Tag Viewport - Essencial para Mobile^[12]

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Sem isso, sites não funcionam corretamente em mobile.^[12]

9. Comparação de Frameworks em 2024^[4]

Framework	Vantagens	Desvantagens
Bootstrap	Componentes prontos, grid flexível, rápido desenvolvimento	Código redundante se não otimizado
Tailwind CSS	Customização alta, controle granular	Curva de aprendizado maior
Foundation	Grid avançado, acessibilidade	Mais complexo para iniciantes

10. Acessibilidade e Desempenho^[13] ^[4]

- **Imagens:** otimize tamanho (sites usam 1,8MB de imagens = 60% do peso)^[13]
- **CDN:** use rede de entrega para arquivos estáticos (CSS, JS, imagens)^[13]
- **Teste:** use Lighthouse do Chrome para verificar performance e acessibilidade^[4]
- **Evite:** divs desnecessárias, CSS inline, posicionamentos absolute/float excessivos^[14]

11. Código HTML Completo para Copiar

O código HTML completo acima contém 10 exemplos práticos:

1. ✓ Header responsivo com Flexbox
2. ✓ Grid layout com sidebar
3. ✓ Cards grid auto-flexível (auto-fit, minmax)
4. ✓ Fontes com clamp()

5. ✓ -unidades relativas (rem, em, %)
6. ✓ Imagens com `srcset` + `sizes`
7. ✓ Vídeo responsivo (aspect ratio 16:9)
8. ✓ Breakpoint tablet (769px-1024px)
9. ✓ Breakpoint desktop grande (1400px+)
10. ✓ Orientação portrait/landscape

Salve como `index.html` e abra no navegador — teste colisando a janela ou usando Device Mode do Chrome (F12).

Resumo Final

Para responsividade eficiente:

1. Use **mobile-first** como abordagem^[4]
2. Combine **CSS Grid** + **Flexbox**^[5] ^[4]
3. Use **unidades relativas** (rem, em, %) ^[7]
4. Implemente **media queries** com breakpoints estratégicos^[4]
5. Otimize **imagens com srcset**^[9] ^[4]
6. Use **clamp()** para textos responsivos^[10]
7. Teste em **dispositivos reais**^[4]

✱

Acompanhamentos

Checklist de implementação: melhores práticas para converter um layout fixo em responsivo com CSS moderno

Computer

Comparativo de performance: Flexbox vs. CSS Grid para layouts complexos em dispositivos móveis e desktops

Computer

Como usar unidades relativas como rem e em para tipografia fluida

Diferenças entre mobile-first e desktop-first no CSS moderno

Melhores práticas para otimizar imagens responsivas com srcset e sizes

Também tenho um banner que fica perfeito no site mas quebra no mobile

Com base na sua pesquisa, vou entregar um **checklist completo** + soluções práticas para cada ponto, incluindo a correção do banner que quebra no mobile:

☒ Checklist de Implementação: Layout Fixo → Responsivo com CSS Moderno

ETAPA 1: Preparação

```
/* 1. Tag viewport essencial */
<meta name="viewport" content="width=device-width, initial-scale=1.0">

/* 2. Reset básico */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* 3. Remover overflow horizontal (problema comum) */
html, body {
  overflow-x: hidden;
  width: 100%;
}
```

ETAPA 2: Converter Medidas Fixas

Tipo	Fixo (errado)	Responsivo (correto)
Largura	width: 1200px	max-width: 1200px; width: 100%
Padding	padding: 50px	padding: 3rem(ou clamp(2rem, 5vw, 5rem))
Margem	margin: 20px	margin: 1.5rem
Fonte	font-size: 16px	font-size: 1rem
Height	height: 600px	min-height: 600px; height: auto

ETAPA 3: Implementar Layout Flexível

```
/* Opção A: CSS Grid (layouts 2D) */
.container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 2rem;
}

/* Opção B: Flexbox (layouts 1D) */
.container {
  display: flex;
  flex-wrap: wrap;
  gap: 2rem;
}
```

ETAPA 4: Adicionar Media Queries

```
/* MOBILE (base - sem media query) */
.container {
  grid-template-columns: 1fr;
}

/* TABLET (+768px) */
@media (min-width: 768px) {
  .container {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* DESKTOP (+1024px) */
@media (min-width: 1024px) {
  .container {
    grid-template-columns: repeat(3, 1fr);
  }
}

/* DESKTOP GRANDE (+1400px) */
@media (min-width: 1400px) {
  .container {
    grid-template-columns: repeat(4, 1fr);
    gap: 3rem;
  }
}
```

☒ Flexbox vs CSS Grid: Comparativo de Performance

Aspecto	Flexbox	CSS Grid
Uso ideal	Layouts 1D (linhas OU colunas)	Layouts 2D (linhas + colunas)
Performance mobile	✔ Levemente mais rápido em layouts simples ^[20]	✔ Quase igual, diferença insignificante
Performance desktop	✔ Igual para layouts simples	✔ Melhor para layouts complexos
Complexidade	Simples, intuitivo	Mais verboso, mas poderoso
Quando usar	Menu, alinhamento horizontal, cards simples	Grid de produtos, layouts com sidebar, páginas completas
Caso de uso	.nav { display: flex; gap: 1rem; }	.page { display: grid; grid-template-columns: 1fr 3fr; }

Regra prática: ^[20] ^[21]

- Use **Flexbox** para: alinhamento de elementos em uma direção
- Use **Grid** para: layouts estruturados com múltiplas colunas/linhas

- Combine os dois: Grid para layout principal, Flexbox para componentes internos

☒ rem vs em: Guia de Tipografia Fluida

Diferença Fundamental^[22] ^[23] ^[24]

```
/* HTML */
<html style="font-size: 16px;">  <!-- root (raiz) -->

<div style="font-size: 20px;">    <!-- pai -->
  <p>Texto</p>
</div>
```

Unidade	Referência	Exemplo	Uso ideal
rem	Fonte do <html> (raiz)	2rem = 32px (se root=16px)	Tipografia global, layout principal
em	Fonte do elemento pai	2em = 40px (se pai=20px)	Componentes internos, botões, ícones

Exemplos Práticos

```
/* ✓ USE rem para tipografia global */
html {
  font-size: 16px; /* base */
}

body {
  font-size: 1rem; /* 16px */
}

h1 {
  font-size: 2.5rem; /* 40px */
}

h2 {
  font-size: 2rem; /* 32px */
}

p {
  font-size: 1rem; /* 16px */
}

/* ✓ USE em para componentes */
.button {
  font-size: 1rem;
  padding: 0.75em 1.5em; /* relativo ao font-size do botão */
}

.icon {
  width: 1.5em; /* relativo ao font-size do pai */
  height: 1.5em;
}
```

```

/* ✓ TIPGRAFIA FLUIDA com clamp() */
.responsive-title {
  font-size: clamp(1.5rem, 3vw, 3rem);
  /* mínimo: 1.5rem, preferencial: 3vw, máximo: 3rem */
}

.responsive-text {
  font-size: clamp(1rem, 0.5vw + 0.5rem, 1.5rem);
}

```

Melhor Prática ^[22]

- Use **rem** para: tudo que deve ser consistente globalmente
- Use **em** para: espaçamentos dentro de componentes (padding, margin)

☒ Mobile-First vs Desktop-First: Diferenças no CSS Moderno

Mobile-First (Abordagem Moderna - Dominante em 2026) ^[25] ^[26]

```

/* Estilos BASE para mobile (sem media query) */
.container {
  grid-template-columns: 1fr;
  padding: 1rem;
}

/* ADICIONAR complexidade com min-width */
@media (min-width: 768px) {
  .container {
    grid-template-columns: repeat(2, 1fr);
    padding: 2rem;
  }
}

@media (min-width: 1024px) {
  .container {
    grid-template-columns: repeat(3, 1fr);
    padding: 3rem;
  }
}

```

Vantagens:

- ✓ Código otimizado para mobile (carga mais rápida)
- ✓ Acessibilidade melhor
- ✓ **abbr:**Progressive enhancement (adiciona features conforme tela cresce)
- ✓ Padrão da indústria em 2026 ^[25]

Desktop-First (Abordagem Antiga)^[26] ^[25]

```
/* Estilos BASE para desktop */
.container {
  grid-template-columns: repeat(3, 1fr);
  padding: 3rem;
}

/* REDUZIR com max-width */
@media (max-width: 1024px) {
  .container {
    grid-template-columns: repeat(2, 1fr);
    padding: 2rem;
  }
}

@media (max-width: 768px) {
  .container {
    grid-template-columns: 1fr;
    padding: 1rem;
  }
}
```

Problemas:

- ✖ Mobile carrega CSS pesado do desktop
- ✖ Desempenho pior em dispositivos móveis
- ✖ Overengineering (precisa "desconstruir" o layout)

Comparação Direta^[26] ^[25]

Aspecto	Mobile-First	Desktop-First
Media query	min-width (aumenta)	max-width (diminui)
CSS base	Mobile (sem query)	Desktop (sem query)
Performance mobile	✔ Otimizado	✖ Carrega tudo
Popularidade 2026	✔ Dominante	✖ Antiga
Quandos breakpoints	3-5 (padrão) ^[25]	3-5

Recomendação: Use mobile-first como padrão. ^[25]

☒ Otimização de Imagens Responsivas com srcset e sizes

Tags Essenciais ^[27] ^[28] ^[29]

```
<!-- IMAGEM RESPONSIVA COM srcset E sizes -->

```

Como Funciona ^[30] ^[27]

Atributo	Função
src	Imagem fallback (usada se srcset não funcionar)
srcset	Lista de imagens por largura (400w, 800w, 1200w...)
sizes	Condicional: qual largura usar em cada breakpoint
width/height	OBRIGATÓRIO para evitar CLS (Layout Shift) ^[30]
loading="lazy"	Carrega apenas quando visível (melhora performance) ^[27]

Exemplo de sizes ^[27]

```
/* sizes explica como a imagem se comporta no layout */

/* Imagem ocupa 100% da tela em mobile */
(max-width: 768px) 100vw

/* Imagem ocupa 50% da tela em tablet */
(max-width: 1024px) 50vw

/* Imagem ocupa 33% em desktop (grid de 3 colunas) */
33vw
```

Melhores Práticas de Otimização ^[31] ^[27]

Prática	Descrição
Formato WebP/AVIF	30% mais leve que JPEG/PNG ^[31]
Compressão	Qualidade 80% (cwebp -q 80) ^[27]

Prática	Descrição
Dimensionamento	Máximo 2x da largura de exibição [27]
Lazy loading	loading="lazy" em imagens abaixo do viewport [27]
Eager loading	loading="eager" + fetchpriority="high" na imagem hero [27]
CDN de imagens	Use Cloudflare, Imgix para resize automático [31]
Audit	PageSpeed Insights para identificar imagens pesadas [27]

Imagem Pesada? Converta para WebP

```
# Com cwebp (CLI)
cwebp -q 80 imagem-jpeg.jpg -o imagem-webp.webp

# Ou use plugin (WordPress, etc)
```

❌ SOLUÇÃO: Banner que Quebra no Mobile

Problema Comum

Banner fixo (ex: width: 1920px, height: 600px) funciona no desktop mas:

- No mobile aparece com zoom
- Cria overflow horizontal
- Corta conteúdo importante

Solução 1: CSS com object-fit (Recomendada) [\[32\]](#) [\[33\]](#) [\[34\]](#) [\[35\]](#)

```
.banner {
  width: 100%;
  height: 600px; /* ou qualquer altura */
  object-fit: cover; /* crops sem distorcer */
  object-position: center; /* centraliza o crop */
}

/* MOBILE: altura menor */
@media (max-width: 768px) {
  .banner {
    height: 300px;
    object-position: bottom; /* foca no conteúdo inferior */
  }
}
```

```

```

Solução 2: background-size + media query^[32]

```
.banner {
    background-image: url('banner-desktop.jpg');
    background-size: cover; /* NÃO usar contain! */
    background-position: center;
    width: 100%;
    height: 600px;
}

/* MOBILE: imagem diferente */
@media (max-width: 768px) {
    .banner {
        background-image: url('banner-mobile.jpg');
        height: 300px;
    }
}
```

Solução 3: Imagens Diferentes (Desktop vs Mobile)^[36]

```
<!-- Desktop: imagem horizontal -->
<div class="banner-desktop">
    
</div>

<!-- Mobile: imagem vertical (aparece só em mobile) -->
<div class="banner-mobile" style="display: none;">
    
</div>

<style>
/* Mobile */
@media (max-width: 768px) {
    .banner-desktop { display: none; }
    .banner-mobile { display: block; }
}
</style>
```

Solução 4: Flexbox + Width Relativo

```
.banner-container {
    display: flex;
    width: 100%;
    max-width: 1200px;
    margin: 0 auto;
```

```

}

.banner {
  width: 100%;
  height: auto;
  max-height: 600px;
}

/* MOBILE */
@media (max-width: 768px) {
  .banner {
    max-height: 300px;
  }
}

```

Checklist para Banner Responsivo

```

/* ✓ CORRETO */
.banner {
  width: 100%;           /* NÃO: 1920px */
  height: auto;          /* OU: height fixo + object-fit */
  max-width: 1200px;     /* limita no desktop */
  object-fit: cover;     /* crops sem distorção */
  overflow: hidden;      /* previne overflow */
}

/* ✓ MOBILE */
@media (max-width: 768px) {
  .banner {
    height: 300px;        /* menor no mobile */
    object-position: bottom; /* foca conteúdo importante */
  }
}

/* ✗ ERRADO (causa quebra) */
.banner {
  width: 1920px;          /* FIXO: quebra no mobile */
  height: 600px;          /* FIXO: sem flexibilidade */
  object-fit: contain;    /* CONTAIN: estica demais */
}

```

📄 Código Completo para Banner

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Banner Responsivo</title>
  <style>
    * {
      margin: 0;

```

```

        padding: 0;
        box-sizing: border-box;
    }

    body {
        overflow-x: hidden;
    }

    .banner {
        width: 100%;
        height: 600px;
        object-fit: cover;
        object-position: center;
        display: block;
    }

    /* MOBILE: altura menor + object-position diferente */
    @media (max-width: 768px) {
        .banner {
            height: 300px;
            object-position: bottom;
        }
    }

    /* TABLET */
    @media (min-width: 769px) and (max-width: 1024px) {
        .banner {
            height: 450px;
        }
    }
</style>
</head>
<body>
    
</body>
</html>

```

Teste: Abra no navegador, diminua a janela e veja o banner adaptar sem quebrar.

Se quiser, posso criar um arquivo HTML completo com todos os exemplos praticados para você testar!

✱✱

Acompanhamentos

Sim, pode criar um HTML completo com esses exemplos para teste

Quero ver como aplicar clamp em tipografia para escalabilidade

Poderia explicar melhor quando usar auto-fit vs auto-fill no Grid

Como resolver o colapso de imagens dentro de containers flexíveis

Mostre como implementar um menu hambúrguer com CSS moderno apenas

Sim — abaixo está um **HTML completo** com os exemplos principais: tipografia fluida com `clamp()`, Grid com auto-fit, imagem em container flexível e menu hambúrguer só com CSS. `clamp()` limita o tamanho entre mínimo e máximo e deixa a tipografia crescer com a viewport.

[\[42\]](#) [\[43\]](#)

HTML completo

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Responsividade CSS Moderna</title>
  <style>
    :root {
      --bg: #0f172a;
      --surface: #111827;
      --surface-2: #1f2937;
      --text: #e5e7eb;
      --muted: #9ca3af;
      --primary: #38bdf8;
      --maxw: 1120px;
      --radius: 16px;
    }

    * {
      box-sizing: border-box;
    }

    html {
      font-size: 16px;
      scroll-behavior: smooth;
    }

    body {
      margin: 0;
      font-family: Inter, system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif;
      background: linear-gradient(180deg, #020617 0%, #0f172a 100%);
      color: var(--text);
```

```
    line-height: 1.5;
}

img {
    max-width: 100%;
    height: auto;
    display: block;
}

a {
    color: inherit;
    text-decoration: none;
}

.wrap {
    width: min(100% - 2rem, var(--maxw));
    margin-inline: auto;
}

.header {
    position: sticky;
    top: 0;
    z-index: 20;
    background: rgba(15, 23, 42, 0.88);
    backdrop-filter: blur(12px);
    border-bottom: 1px solid rgba(148, 163, 184, 0.15);
}

.header__inner {
    min-height: 72px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 1rem;
    padding-block: .75rem;
}

.brand {
    font-size: clamp(1.1rem, 1vw + 1rem, 1.6rem);
    font-weight: 800;
    letter-spacing: .02em;
}

.nav {
    display: flex;
    gap: 1rem;
    align-items: center;
}

.nav a {
    color: var(--muted);
    font-size: 0.95rem;
    padding: .5rem .75rem;
    border-radius: 999px;
}
```

```
.nav a:hover {
  color: var(--text);
  background: rgba(255,255,255,.06);
}

.hero {
  padding: clamp(3rem, 6vw, 7rem) 0 2rem;
}

.hero h1 {
  font-size: clamp(2rem, 4vw, 4.5rem);
  line-height: 1.05;
  margin: 0;
  max-width: 12ch;
}

.hero p {
  font-size: clamp(1rem, 1vw + .9rem, 1.3rem);
  color: var(--muted);
  max-width: 65ch;
  margin-top: 1rem;
}

.grid-demo {
  padding: 2rem 0 4rem;
}

.section-title {
  font-size: clamp(1.4rem, 2vw, 2.1rem);
  margin: 0 0 1rem;
}

.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}

.card {
  background: rgba(17, 24, 39, .9);
  border: 1px solid rgba(148, 163, 184, .15);
  border-radius: var(--radius);
  padding: 1.2rem;
  min-height: 180px;
}

.card h3 {
  font-size: clamp(1.05rem, 1vw + .9rem, 1.35rem);
  margin: 0 0 .5rem;
}

.card p {
  color: var(--muted);
  margin: 0;
  font-size: 1rem;
}
```

```
.media {
  padding: 0 0 4rem;
}

.flex-box {
  display: flex;
  gap: 1rem;
  align-items: center;
  justify-content: space-between;
  flex-wrap: wrap;
  background: rgba(17, 24, 39, .9);
  border: 1px solid rgba(148, 163, 184, .15);
  border-radius: var(--radius);
  padding: 1rem;
}

.media__image {
  flex: 1 1 320px;
  min-width: 0;
}

.media__image img {
  width: 100%;
  aspect-ratio: 16 / 9;
  object-fit: cover;
  object-position: center;
  border-radius: 12px;
}

.media__text {
  flex: 1 1 280px;
  min-width: 0;
}

.media__text p {
  color: var(--muted);
  margin: .5rem 0 0;
}

.menu-toggle {
  display: none;
}

.burger {
  display: none;
  width: 44px;
  height: 44px;
  border-radius: 12px;
  align-items: center;
  justify-content: center;
  cursor: pointer;
  background: rgba(255,255,255,.06);
}

.burger span,
```



```

.burger span::before,
.burger span::after {
  content: "";
  display: block;
  width: 22px;
  height: 2px;
  background: var(--text);
  border-radius: 999px;
  position: relative;
}

.burger span::before {
  position: absolute;
  top: -7px;
  left: 0;
}

.burger span::after {
  position: absolute;
  top: 7px;
  left: 0;
}

@media (max-width: 768px) {
  .burger {
    display: inline-flex;
  }

  .nav {
    display: none;
    position: absolute;
    left: 1rem;
    right: 1rem;
    top: 72px;
    flex-direction: column;
    align-items: stretch;
    gap: .25rem;
    padding: .75rem;
    background: #0b1220;
    border: 1px solid rgba(148, 163, 184, .15);
    border-radius: 16px;
  }

  .nav a {
    width: 100%;
    padding: .85rem 1rem;
  }

  .menu-toggle:checked ~ .nav {
    display: flex;
  }
}
</style>
</head>
<body>
  <header class="header">

```

```

<div class="wrap header__inner">
  <a class="brand" href="#">Site Responsivo</a>

  <input class="menu-toggle" type="checkbox" id="menu-toggle">
  <label class="burger" for="menu-toggle" aria-label="Abrir menu">
    <span></span>
  </label>

  <nav class="nav" aria-label="Navegação principal">
    <a href="#tipografia">Tipografia</a>
    <a href="#grid">Grid</a>
    <a href="#imagem">Imagem</a>
  </nav>
</div>
</header>

<main>
  <section class="hero wrap" id="tipografia">
    <h1>Tipografia fluida, grid moderno e navegação responsiva</h1>
    <p>
      Este exemplo mostra como aplicar clamp() em títulos e textos, como usar auto-fit para evitar o
      como evitar colapso de imagens dentro de flex containers e como criar um menu responsivo.
    </p>
  </section>

  <section class="grid-demo wrap" id="grid">
    <h2 class="section-title">Grid com auto-fit</h2>
    <div class="cards">
      <article class="card">
        <h3>Card 1</h3>
        <p>auto-fit colapsa colunas vazias e faz os itens ocuparem melhor o espaço.</p>
      </article>
      <article class="card">
        <h3>Card 2</h3>
        <p>Excelente para galerias, produtos e dashboards responsivos.</p>
      </article>
      <article class="card">
        <h3>Card 3</h3>
        <p>Combina muito bem com minmax(220px, 1fr) para fluidez.</p>
      </article>
    </div>
  </section>

  <section class="media wrap" id="imagem">
    <h2 class="section-title">Imagem em container flexível</h2>
    <div class="flex-box">
      <div class="media__image">
        
      </div>
      <div class="media__text">
        <h3 style="margin:0; font-size: clamp(1.05rem, 1vw + .9rem, 1.4rem);">Evitar o colapso de imagens
        <p>
          Em containers flexíveis, o truque é usar width: 100%, max-width: 100%, height: auto.
        </p>
      </div>
    </div>
  </section>

```

```
        min-width: 0 no item flex e, quando necessário, aspect-ratio + object-fit;
    </p>
</div>
</div>
</section>
</main>
</body>
</html>
```

Tipografia com clamp

`clamp()` é ideal quando você quer que a fonte cresça com a tela, mas sem ficar pequena demais no mobile nem gigante no desktop. ^[43] ^[42]

Exemplo prático:

```
h1 {
  font-size: clamp(2rem, 4vw, 4.5rem);
}

p {
  font-size: clamp(1rem, 1vw + .9rem, 1.25rem);
}
```

A lógica é: mínimo, valor fluido e máximo. ^[42]

Na prática, isso reduz a necessidade de vários breakpoints só para tipografia. ^[43]

auto-fit vs auto-fill

`auto-fit` e `auto-fill` parecem parecidos, mas a diferença aparece quando sobram espaços no grid. ^[44] ^[45]

Com `auto-fit`, as trilhas vazias colapsam e os itens existentes se expandem; com `auto-fill`, as trilhas vazias continuam reservadas. ^[45]

Use assim:

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
}
```

Use `auto-fit` quando você quer que os cards “estiquem” e preencham melhor a linha.

Use `auto-fill` quando você quer manter a estrutura de colunas mesmo se alguns slots ficarem vazios, o que pode ser útil em galerias e layouts que dependem de alinhamento constante. ^[44] ^[45]

Imagens em flex

O colapso de imagens em flex containers normalmente acontece por restrições de largura, ausência de `min-width: 0` no item flex ou imagem sem regras fluidas.

A base segura é: `img { max-width: 100%; height: auto; display: block; }` e, no filho flex, `min-width: 0` para permitir encolhimento.

Quando precisar de corte controlado sem deformar, use:

```
img {  
  width: 100%;  
  aspect-ratio: 16 / 9;  
  object-fit: cover;  
  object-position: center;  
}
```

Isso preserva proporção e evita que a imagem “quebre” o layout em telas pequenas.

Menu hambúrguer só CSS

O padrão mais simples e robusto é `checkbox + label + :checked`, sem JavaScript. [\[46\]](#) [\[47\]](#) [\[48\]](#)

Ele funciona bem para menus compactos e é fácil de manter.

Estrutura mínima:

```
<input type="checkbox" id="menu-toggle">  
<label for="menu-toggle">Menu</label>  
<nav class="nav">...</nav>
```

E o CSS:

```
.nav { display: none; }  
#menu-toggle:checked ~ .nav { display: flex; }  
  
@media (min-width: 769px) {  
  .nav { display: flex; }  
}
```

Próximo passo

O HTML acima já serve como base de teste local.

O próximo ajuste mais valioso é adaptar o banner do seu site com `aspect-ratio`, `object-fit` e uma imagem específica para mobile quando o recorte do desktop não funcionar bem. [\[49\]](#) [\[50\]](#)

Acompanhamentos

Como evitar que o uso de clamp em tipografia prejudique a acessibilidade e o zoom do navegador

Pesquisa profunda

Como usar aspect-ratio para imagens responsivas sem distorção

Melhores práticas de acessibilidade para menus responsivos CSS

Diferenças entre object-fit contain e cover no layout mobile

Como evitar overflow horizontal com elementos de largura fixa

Sim — e há alguns cuidados importantes para não perder acessibilidade quando usa `clamp()` e para evitar problemas com imagens e menus responsivos. `clamp()` é seguro para tipografia fluida, mas você precisa combinar o valor preferido com unidades relativas e testar zoom de 200% para não travar a escala do texto. ^[54] ^[55]

Tipografia com clamp

O risco de acessibilidade aparece quando você usa só `vw` no valor preferido, porque o texto pode ficar muito dependente da largura da viewport e responder mal ao zoom. ^[55] ^[54]

A prática mais segura é usar `rem` no mínimo e no máximo, com um valor preferido que ainda permita crescimento fluido, por exemplo `clamp(1rem, 1rem + 1vw, 2rem)`. ^[54] ^[55]

```
h1 {
  font-size: clamp(2rem, 1.2rem + 2vw, 4rem);
  line-height: 1.1;
}

p {
  font-size: clamp(1rem, 0.95rem + 0.4vw, 1.2rem);
  line-height: 1.6;
}
```

O ponto principal é manter o texto dentro de limites legíveis e garantir que ele ainda possa crescer bem quando o usuário der zoom no navegador. ^[55] ^[54]

Zoom e acessibilidade

Para não prejudicar o zoom, evite fontes mínimas muito baixas e não “congele” a tipografia com valores exclusivamente baseados em viewport. ^[54] ^[55]

Também vale testar o layout em 200% de zoom e checar se a linha continua legível, sem estourar container nem provocar corte de conteúdo. ^[54]

Uma boa heurística é manter o texto do corpo em `rem`, e usar `clamp()` só para escalar títulos, subtítulos e chamadas visuais.

Se a fonte principal depender demais de `vw`, o layout pode parecer bonito no desktop, mas quebrar em zoom ou em telas pequenas. ^[55]

Aspect ratio em imagens

`aspect-ratio` ajuda a reservar espaço antes da imagem carregar e a manter a proporção correta em containers responsivos. ^[56]

Em elementos substituídos como imagens e vídeo, o comportamento padrão já respeita a proporção intrínseca, mas `aspect-ratio` dá controle mais previsível quando o container é flexível. ^[56]

```
.hero-img {  
  width: 100%;  
  aspect-ratio: 16 / 9;  
  object-fit: cover;  
  object-position: center;  
  display: block;  
}
```

Se a imagem estiver dentro de um bloco com largura fluida, essa abordagem evita distorção e reduz layout shift. ^[56]

Se quiser mostrar a imagem inteira sem corte, troque o `object-fit` para `contain`. ^[57] ^[58]

contain vs cover

`cover` preenche totalmente o container, mas pode cortar partes da imagem. ^[58] ^[57]

`contain` mostra a imagem inteira, mas pode deixar áreas vazias dentro do box. ^[57] ^[58]

Valor	Resultado	Quando usar
<code>cover</code>	Preenche o container e corta excedentes	Hero, banner, cards com recorte controlado
<code>contain</code>	Mantém a imagem inteira e pode sobrar espaço	Logotipos, artes, imagens que não podem ser cortadas

No mobile, `cover` é ótimo quando a prioridade é preencher o espaço com impacto visual.

Já `contain` é melhor quando cortar o conteúdo seria um problema, como em logos, selos ou banners com texto embutido. ^[58] ^[57]

Menus acessíveis

Menus responsivos só com CSS precisam continuar usáveis por teclado, porque acessibilidade exige operação via teclado. ^[59]

A melhor base é usar um `input checkbox` ou `details/summary`, mantendo foco visível e ordem lógica de navegação. ^[59]

```
<input type="checkbox" id="menu-toggle" class="menu-toggle">  
<label for="menu-toggle" class="menu-button">Menu</label>  
  
<nav class="menu">  
  <a href="#">Início</a>  
  <a href="#">Serviços</a>  
  <a href="#">Contato</a>  
</nav>
```

```

.menu-toggle {
  position: absolute;
  opacity: 0;
  pointer-events: none;
}

.menu-button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  padding: .75rem 1rem;
  border-radius: .75rem;
}

.menu a:focus-visible,
.menu-button:focus-visible {
  outline: 2px solid currentColor;
  outline-offset: 3px;
}

.menu {
  display: none;
}

.menu-toggle:checked ~ .menu {
  display: flex;
}

```

Se o item for clicável, ele também precisa ser focável e operar via teclado; e o foco precisa ser claramente visível. ^[59]

Se o menu tiver conteúdo rolável, pense também em navegação por teclado dentro dele e em não esconder o foco com `overflow: hidden` sem necessidade. ^[60] ^[59]

Overflow horizontal

Overflow horizontal costuma vir de largura fixa em elementos como banners, imagens, blocos e tabelas.

A primeira defesa é evitar `width: 1200px` e preferir `max-width: 100%` ou `width: min(100%, 1200px)`.

```

img, video, iframe {
  max-width: 100%;
  height: auto;
}

.container {
  width: min(100% - 2rem, 1200px);
  margin-inline: auto;
}

```

Outro ponto útil é usar `min-width: 0` em filhos de flex/grid que precisam encolher, porque alguns itens não diminuem como o esperado sem isso.

Se houver conteúdo realmente maior que o container, `overflow-x: auto` é melhor do que esconder tudo, porque mantém acessibilidade e permite acesso ao conteúdo. ^[61] ^[60]

Resumo prático

Para manter acessibilidade com `clamp()`, use valores mínimos e máximos em `rem` e teste zoom real. ^[55] ^[54]

Para imagens, combine `aspect-ratio`, `width: 100%` e `object-fit` conforme a intenção visual. ^[57] ^[56]

Para menus, garanta foco visível e navegação por teclado. ^[59]

Para overflow horizontal, remova larguras fixas e use limites fluidos com `max-width` e `min-width: 0`. ^[61]

Posso montar agora um **HTML único de laboratório responsivo** com esses quatro temas aplicados em blocos separados.

*
**

Acompanhamentos

Como `aspect-ratio` evita layout shift com imagens de tamanhos variáveis
Diferenças entre unidades `rem` e `em` na escala de fontes responsivas
Como testar conformidade WCAG de tipografia em navegadores modernos
Boas práticas de `line-height` com `clamp` para manter a legibilidade
Limitações de `aspect-ratio` em containers com conteúdo de texto dinâmico

Sim — abaixo está a leitura prática desses pontos, com foco em CSS moderno e validação de acessibilidade. `aspect-ratio` ajuda a reservar espaço antes da imagem carregar, reduzindo layout shift, e `rem` costuma ser a base mais segura para escala global de fontes. ^[64] ^[65]

Aspect-ratio e layout shift

`aspect-ratio` permite que o navegador calcule a proporção do bloco antes da mídia carregar, então o espaço já fica reservado e o conteúdo abaixo não “salta”. ^[66] ^[64]

Isso é especialmente útil em imagens responsivas e cards com thumbnails, porque evita reflow quando a imagem finalmente pinta. ^[64]

```
.card__image {  
  width: 100%;  
  aspect-ratio: 16 / 9;  
  object-fit: cover;  
}
```

Se você já tem `width` e `height` no `img`, o navegador também consegue inferir a proporção e reduzir layout shift. ^[66]

`aspect-ratio` vira ainda mais útil quando a imagem entra num container flexível, porque ele estabiliza o bloco mesmo antes do download. ^[67] ^[66]

Rem e em

`rem` é relativo ao `html`, então é ótimo para tipografia global porque não sofre com a multiplicação de tamanhos em níveis internos. ^[65] ^[68]

`em` é relativo ao elemento pai, o que é útil quando você quer que um componente escale junto com o tamanho da fonte dele mesmo, como botões, badges e ícones. ^[68] ^[69]

Unidade	Base	Melhor uso
<code>rem</code>	raiz (<code>html</code>)	tipografia global, espaçamentos consistentes
<code>em</code>	pai imediato	componentes internos, padding proporcional, ícones

Na prática, `rem` simplifica o controle da escala inteira do site, enquanto `em` é bom para comportamento local e modular. ^[65] ^[68]

WCAG e tipografia

Para checar conformidade de tipografia, o ponto mais conhecido é o WCAG 1.4.12 Text Spacing, que exige que o conteúdo continue legível e funcional com espaçamentos aumentados, incluindo `line-height` de até 1.5. ^[70] ^[71]

O jeito mais confiável de testar em navegadores modernos é aplicar manualmente esses valores e verificar se nada quebra: `line-height: 1.5`, `letter-spacing` de 0.12em, `word-spacing` de 0.16em e `paragraph spacing` de 2em. ^[71] ^[70]

```
body {  
  line-height: 1.5;  
}  
  
p {  
  margin-bottom: 2em;  
}
```

O ideal é testar com zoom do navegador, aumento de texto e extensões ou ferramentas de auditoria que simulam as exigências do WCAG. ^[72] ^[71]

Também vale observar comprimento de linha, porque linhas muito longas ou muito curtas prejudicam leitura e ritmo visual. ^[70]

Line-height com clamp

`clamp()` pode funcionar bem para `font-size`, mas o `line-height` deve permanecer estável o bastante para preservar legibilidade. ^[73] ^[70]

Uma boa prática é usar `clamp()` no tamanho da fonte e manter `line-height` unitless, por exemplo 1.5, porque isso escala com o tamanho do texto sem precisar de ajuste fino por breakpoint. ^[70]

```
h1 {  
  font-size: clamp(2rem, 1.2rem + 2vw, 4rem);  
  line-height: 1.1;  
}
```

```
p {
  font-size: clamp(1rem, 0.95rem + 0.4vw, 1.2rem);
  line-height: 1.6;
}
```

Para corpo de texto, 1.5 é um ponto de partida seguro e amplamente suportado em acessibilidade. ^[71] ^[70]

Se a fonte cresce muito em telas grandes, a linha deve acompanhar com cuidado para não ficar apertada nem excessivamente espaçada. ^[70]

Limites do aspect-ratio

`aspect-ratio` funciona muito bem para mídia e blocos visuais, mas não resolve conteúdo de texto dinâmico sozinho. ^[74] ^[64]

Se um card tiver texto variável, o conteúdo pode ultrapassar a altura prevista, então o layout precisa de `min-height`, padding adaptável e, às vezes, `max-width` ou `line-clamp` para controlar a densidade. ^[74]

```
.card {
  aspect-ratio: 4 / 3;
  display: flex;
  flex-direction: column;
  min-height: 0;
}

.card__content {
  overflow: auto;
}
```

Em resumo, `aspect-ratio` é ótimo para reservar espaço de imagens e vídeos, mas não deve ser usado como única regra para containers com texto imprevisível. ^[64] ^[74]

Recomendações práticas

Para um layout acessível e estável, use `rem` para a base tipográfica, `em` para componentes, `clamp()` com parcimônia e `line-height` unitless. ^[68] ^[65] ^[70]

Para imagens, combine `width`, `height` ou `aspect-ratio` para prevenir layout shift. ^[66] ^[64]

Para validar WCAG, teste zoom, espaçamento de texto e navegação real no teclado em navegadores modernos. ^[75] ^[71]

Se quiser, eu posso montar um **HTML de laboratório** com exemplos lado a lado de `rem`, `em`, `clamp()`, `aspect-ratio` e um bloco de testes WCAG.

✱✱

Acompanhamentos

Sim, crie o HTML de laboratório com todos esses exemplos aplicados

Por favor, monte esse HTML de testes com foco na validação WCAG

Pode criar o HTML focando nos problemas de layout shift com imagens

Gostaria de ver esse HTML, especialmente como rem e em se comportam

Sim, crie o HTML, mas inclua também exemplos de erro para comparar

Perfeito — abaixo está um **HTML de laboratório completo**, já com **exemplos corretos e erros intencionais** para comparar. Ele foi montado para validar: WCAG de tipografia, comportamento de rem e em, prevenção de CLS com imagens, aspect-ratio, e diferenças práticas entre layouts bons e ruins. [\[77\]](#) [\[78\]](#) [\[79\]](#)

HTML de laboratório

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Laboratório Responsivo e WCAG</title>
  <style>
    :root {
      --bg: #0b1020;
      --surface: #141b2d;
      --surface-2: #1f2a44;
      --text: #e8eeefc;
      --muted: #a9b5d0;
      --accent: #7dd3fc;
      --danger: #fb7185;
      --ok: #4ade80;
      --max: 1200px;
      --radius: 16px;
    }

    * {
      box-sizing: border-box;
    }

    html {
      font-size: 100%;
      scroll-behavior: smooth;
    }

    body {
      margin: 0;
      background: linear-gradient(180deg, #060912 0%, #0b1020 100%);
      color: var(--text);
      font-family: Inter, system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif;
      line-height: 1.5;
    }

    img, video {
```

```
    max-width: 100%;
    display: block;
}

a {
    color: inherit;
}

.wrap {
    width: min(100% - 2rem, var(--max));
    margin-inline: auto;
}

.hero {
    padding: 2rem 0 1rem;
}

.hero h1 {
    font-size: clamp(2rem, 1.2rem + 3vw, 4.5rem);
    line-height: 1.08;
    margin: 0;
}

.hero p {
    max-width: 70ch;
    font-size: clamp(1rem, 0.95rem + 0.35vw, 1.2rem);
    color: var(--muted);
}

.grid {
    display: grid;
    gap: 1rem;
}

.two {
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
}

.card {
    background: rgba(20, 27, 45, .92);
    border: 1px solid rgba(125, 211, 252, .15);
    border-radius: var(--radius);
    padding: 1rem;
}

.card h2, .card h3 {
    margin: 0 0 .75rem;
}

.tag {
    display: inline-block;
    padding: .25rem .6rem;
    border-radius: 999px;
    font-size: .85rem;
    background: rgba(125, 211, 252, .12);
    color: var(--accent);
}
```

```
    margin-bottom: .75rem;
}

.good {
  border-left: 4px solid var(--ok);
}

.bad {
  border-left: 4px solid var(--danger);
}

.typography-sample p {
  margin: 0 0 1rem;
}

.text-rem {
  font-size: 1rem;
}

.text-em {
  font-size: 1em;
}

.parent-rem {
  font-size: 20px;
}

.parent-rem .child-em {
  font-size: 1.5em;
}

.parent-rem .child-rem {
  font-size: 1.5rem;
}

.spaced {
  line-height: 1.6;
}

.bad-typing {
  font-size: clamp(12px, 2vw, 18px);
  line-height: 1.1;
}

.img-good {
  width: 100%;
  aspect-ratio: 16 / 9;
  object-fit: cover;
  object-position: center;
  border-radius: 12px;
  background: #0f172a;
}

.img-bad {
  width: 100%;
  height: 320px;
```

```
    object-fit: contain;
    border-radius: 12px;
    background: #0f172a;
}

.frame {
    background: #0f172a;
    border-radius: 12px;
    overflow: hidden;
}

.fixed-overflow {
    width: 980px;
    max-width: 100%;
    background: rgba(251, 113, 133, .08);
    border: 1px solid rgba(251, 113, 133, .25);
    padding: 1rem;
}

.safe-overflow {
    width: min(100%, 980px);
    background: rgba(74, 222, 128, .08);
    border: 1px solid rgba(74, 222, 128, .25);
    padding: 1rem;
}

.buttons {
    display: flex;
    gap: .75rem;
    flex-wrap: wrap;
}

.btn {
    font-size: 1rem;
    padding: .8em 1.2em;
    border: 0;
    border-radius: 999px;
    background: var(--accent);
    color: #03111f;
    font-weight: 700;
}

.btn-bad {
    font-size: 14px;
    padding: 12px 18px;
    border: 0;
    border-radius: 999px;
    background: #fda4af;
    color: #111827;
    font-weight: 700;
}

.note {
    color: var(--muted);
    font-size: .95rem;
}
```

```

    .split {
      display: grid;
      gap: 1rem;
    }

    @media (min-width: 900px) {
      .split {
        grid-template-columns: 1fr 1fr;
      }
    }
  }
</style>
</head>
<body>
  <main class="wrap">
    <section class="hero">
      <h1>Laboratório de responsividade, acessibilidade e CLS</h1>
      <p>
        Este arquivo compara padrões corretos e erros comuns para validar tipografia
        imagens sem layout shift, escalabilidade com rem/em e organização de layout.
      </p>
    </section>

    <section class="grid two">
      <article class="card good typography-sample">
        <span class="tag">TIPOGRAFIA OK</span>
        <h2>Base com rem + clamp</h2>
        <p class="text-rem spaced">
          Este texto usa rem para respeitar a escala do documento e line-height está
        </p>
        <p class="spaced" style="font-size: clamp(1rem, 0.95rem + 0.4vw, 1.2rem); li
          0 tamanho cresce com a viewport, mas permanece dentro de limites seguros pa
        </p>
      </article>

      <article class="card bad typography-sample">
        <span class="tag">TIPOGRAFIA RUIM</span>
        <h2>Exemplo com risco de acessibilidade</h2>
        <p class="bad-typing">
          Texto com clamp muito agressivo e line-height baixo pode ficar difícil de
        </p>
        <p style="font-size: 12px; line-height: 1.05;">
          Evite dependência excessiva de vw e fontes mínimas baixas.
        </p>
      </article>
    </section>

    <section class="grid two" style="margin-top: 1rem;">
      <article class="card good">
        <span class="tag">REM vs EM</span>
        <h3>Comportamento em cascata</h3>
        <div class="parent-rem">
          <p class="child-em">Este texto usa 1.5em dentro de um pai com 20px.</p>
          <p class="child-rem">Este texto usa 1.5rem e ignora o tamanho do pai.</p>
        </div>
        <p class="note">

```

```

    Use rem para escala global e em para escala local dentro de componentes.
  </p>
</article>

<article class="card bad">
  <span class="tag">EXEMPLO DE ERRO</span>
  <h3>Escala difícil de controlar</h3>
  <div style="font-size: 24px;">
    <p style="font-size: 2em;">Este 2em cresce em cascata conforme o pai muda.</p>
    <p style="font-size: 2em;">Se o componente for aninhado, o tamanho pode mudar.</p>
  </div>
</article>
</section>

<section class="split" style="margin-top: 1rem;">
  <article class="card good">
    <span class="tag">IMAGEM SEM CLS</span>
    <h3>Aspect-ratio + object-fit cover</h3>
    <div class="frame">
      
    </div>
    <p class="note">
      O navegador reserva o espaço antes da imagem carregar, reduzindo layout shift.
    </p>
  </article>

  <article class="card bad">
    <span class="tag">CLS PROVÁVEL</span>
    <h3>Imagem sem proporção previsível</h3>
    <div class="frame">
      
    </div>
    <p class="note">
      Sem largura/altura ou aspect-ratio, o layout pode saltar quando a imagem carregar.
    </p>
  </article>
</section>

<section class="grid two" style="margin-top: 1rem;">
  <article class="card good">
    <span class="tag">LARGURA FLUIDA</span>
    <h3>Sem overflow horizontal</h3>
    <div class="safe-overflow">
      Este bloco usa width fluida e limita o tamanho com min() / max-width.
    </div>
  </article>

```



```

<article class="card bad">
  <span class="tag">ERRO COMUM</span>
  <h3>Elemento fixo demais</h3>
  <div class="fixed-overflow">
    Este bloco simula um elemento muito largo. Em telas pequenas, isso tende a
  </div>
</article>
</section>

<section class="grid two" style="margin: 1rem 0 3rem;">
  <article class="card good">
    <span class="tag">CONTROLES OK</span>
    <h3>Botões legíveis e escaláveis</h3>
    <div class="buttons">
      <button class="btn">Ação principal</button>
      <button class="btn">Outra ação</button>
    </div>
  </article>

  <article class="card bad">
    <span class="tag">CONTROLES RUINS</span>
    <h3>Tamanho fixo rígido</h3>
    <div class="buttons">
      <button class="btn-bad">Ação principal</button>
      <button class="btn-bad">Outra ação</button>
    </div>
  </article>
</section>
</main>
</body>
</html>

```

O que esse laboratório mostra

- `rem` mantendo escala consistente no documento, enquanto `em` herda do pai e pode multiplicar em cascata. ^[78] ^[80]
- `clamp()` em títulos e parágrafos sem exagerar no uso de `vw`, o que ajuda a preservar legibilidade e zoom. ^[81] ^[79] ^[77]
- Imagens com `width/height` e `aspect-ratio` para reservar espaço e reduzir layout shift. ^[82] ^[83] ^[84]
- Exemplos bons e ruins lado a lado para comparar comportamento visual e risco de quebra. ^[85] ^[79]

Como testar no navegador

- Aumente o zoom para 200% e observe se o texto ainda cabe e continua legível. ^[81] ^[77]
- Force espaçamento de texto maior e veja se os blocos continuam funcionais. ^[77] ^[85]
- Reduza a largura da janela para verificar se o bloco de imagem mantém espaço reservado e não “salta”. ^[83] ^[82]

- Compare o comportamento de `rem` e `em` alterando o `font-size` do pai em ferramentas de desenvolvimento. ^[80] ^[78]

Ajustes que valem a pena

Se você quiser deixar esse laboratório ainda mais forte, o próximo passo é adicionar:

- um bloco com `picture + srcset` para testar breakpoints de imagem,
- um menu hambúrguer acessível com foco visível,
- e uma seção de cards com `auto-fit` e `auto-fill` lado a lado.

Posso montar essa próxima versão com esses três blocos extras.

Acompanhamentos

Pode incluir o menu hambúrguer com foco visível e acessível agora

Adicione os blocos de breakpoints de imagem e cards lado a lado sim

Sim, inclua os três blocos extras focando em acessibilidade total

Prefiro ver primeiro como testar o foco do menu com teclado no código

Sim, monte a versão com os três extras e explique as regras WCAG

Sim — abaixo está a **versão expandida** do laboratório com os três blocos extras: menu

hambúrguer acessível com foco visível, breakpoints de imagem com `<picture>`, e cards lado a

lado com auto-fit e auto-fill. O foco do menu pode ser testado só com teclado usando Tab,

Shift+Tab e Enter/Espaço, e todo elemento interativo precisa mostrar foco visível. ^[90] ^[91] ^[92]

Laboratório completo

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Laboratório WCAG, Imagens e Grid</title>
  <style>
    :root {
      --bg: #07111f;
      --surface: #101b2f;
      --surface-2: #17253d;
      --text: #edf3ff;
      --muted: #a9b6d3;
      --accent: #7dd3fc;
      --ok: #4ade80;
      --warn: #fb7185;
      --ring: #fbbf24;
      --max: 1200px;
      --radius: 16px;
    }
  </style>
</head>
<body>
```

```
* { box-sizing: border-box; }

html { font-size: 100%; }

body {
  margin: 0;
  font-family: Inter, system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif;
  background: linear-gradient(180deg, #040816, #07111f);
  color: var(--text);
  line-height: 1.5;
}

img, video {
  max-width: 100%;
  display: block;
}

a {
  color: inherit;
  text-decoration: none;
}

.wrap {
  width: min(100% - 2rem, var(--max));
  margin-inline: auto;
}

.header {
  position: sticky;
  top: 0;
  z-index: 20;
  background: rgba(7, 17, 31, .9);
  backdrop-filter: blur(12px);
  border-bottom: 1px solid rgba(125, 211, 252, .14);
}

.header__inner {
  min-height: 72px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  position: relative;
}

.brand {
  font-weight: 800;
  font-size: clamp(1.1rem, 1vw + 1rem, 1.6rem);
}

.menu-toggle {
  position: absolute;
  opacity: 0;
  pointer-events: none;
}
```

```
.burger {
  width: 48px;
  height: 48px;
  border-radius: 14px;
  display: none;
  align-items: center;
  justify-content: center;
  background: rgba(255,255,255,.06);
  cursor: pointer;
}

.burger span,
.burger span::before,
.burger span::after {
  content: "";
  display: block;
  width: 22px;
  height: 2px;
  background: var(--text);
  border-radius: 999px;
  position: relative;
}

.burger span::before {
  position: absolute;
  top: -7px;
  left: 0;
}

.burger span::after {
  position: absolute;
  top: 7px;
  left: 0;
}

.nav {
  display: flex;
  align-items: center;
  gap: .5rem;
}

.nav a {
  padding: .55rem .85rem;
  border-radius: 999px;
  color: var(--muted);
}

.nav a:hover {
  background: rgba(255,255,255,.06);
  color: var(--text);
}

.nav a:focus-visible,
.burger:focus-visible {
  outline: 3px solid var(--ring);
  outline-offset: 3px;
}
```

```
}

.hero {
  padding: 2rem 0 1rem;
}

.hero h1 {
  margin: 0;
  line-height: 1.08;
  font-size: clamp(2rem, 1.2rem + 3vw, 4.5rem);
  max-width: 12ch;
}

.hero p {
  color: var(--muted);
  font-size: clamp(1rem, 0.95rem + .35vw, 1.2rem);
  max-width: 70ch;
}

.grid {
  display: grid;
  gap: 1rem;
}

.two {
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
}

.card {
  background: rgba(16, 27, 47, .93);
  border: 1px solid rgba(125, 211, 252, .14);
  border-radius: var(--radius);
  padding: 1rem;
}

.card h2, .card h3 { margin-top: 0; }

.tag {
  display: inline-block;
  padding: .25rem .6rem;
  border-radius: 999px;
  font-size: .84rem;
  margin-bottom: .75rem;
  background: rgba(125, 211, 252, .12);
  color: var(--accent);
}

.good { border-left: 4px solid var(--ok); }
.bad { border-left: 4px solid var(--warn); }

.spaced {
  line-height: 1.6;
  margin: 0 0 1rem;
}

.text-fluid {
```

```
    font-size: clamp(1rem, 0.95rem + .4vw, 1.2rem);
    line-height: 1.6;
}

.text-risk {
    font-size: clamp(12px, 2vw, 18px);
    line-height: 1.1;
}

.parent-rem {
    font-size: 20px;
}

.parent-rem .child-em { font-size: 1.5em; }
.parent-rem .child-rem { font-size: 1.5rem; }

.img-shell {
    background: #0b1220;
    border-radius: 12px;
    overflow: hidden;
}

.img-ok {
    width: 100%;
    aspect-ratio: 16 / 9;
    object-fit: cover;
    object-position: center;
}

.img-bad {
    width: 100%;
    height: 300px;
    object-fit: contain;
    background: #0b1220;
}

.flow {
    display: flex;
    gap: 1rem;
    flex-wrap: wrap;
    align-items: flex-start;
}

.safe-box {
    width: min(100%, 980px);
    padding: 1rem;
    background: rgba(74, 222, 128, .08);
    border: 1px solid rgba(74, 222, 128, .22);
}

.fixed-box {
    width: 980px;
    max-width: 100%;
    padding: 1rem;
    background: rgba(251, 113, 133, .08);
    border: 1px solid rgba(251, 113, 133, .22);
}
```

```

}

.buttons {
  display: flex;
  gap: .75rem;
  flex-wrap: wrap;
}

.btn {
  font-size: 1rem;
  padding: .8em 1.2em;
  border: 0;
  border-radius: 999px;
  background: var(--accent);
  color: #06101d;
  font-weight: 800;
}

.btn-bad {
  font-size: 14px;
  padding: 12px 18px;
  border: 0;
  border-radius: 999px;
  background: #fda4af;
  color: #111827;
  font-weight: 700;
}

.sample {
  border: 1px dashed rgba(125, 211, 252, .2);
  border-radius: 12px;
  padding: 1rem;
  background: rgba(255,255,255,.02);
}

.cards-fit {
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
}

.cards-fill {
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
}

.focus-tip {
  color: var(--muted);
  font-size: .95rem;
}

@media (max-width: 768px) {
  .burger { display: inline-flex; }

  .nav {
    display: none;
    position: absolute;
    top: 76px;
    left: 0;
  }
}

```

```

    right: 0;
    flex-direction: column;
    align-items: stretch;
    padding: .75rem;
    gap: .25rem;
    background: #08111f;
    border: 1px solid rgba(125, 211, 252, .14);
    border-radius: 16px;
  }

  .nav a {
    padding: .9rem 1rem;
  }

  .menu-toggle:checked ~ .nav {
    display: flex;
  }
}

@media (prefers-reduced-motion: no-preference) {
  .nav {
    transition: .18s ease;
  }
}
</style>
</head>
<body>
  <header class="header">
    <div class="wrap header__inner">
      <a class="brand" href="#">Laboratório WCAG</a>

      <input class="menu-toggle" type="checkbox" id="menu-toggle">
      <label class="burger" for="menu-toggle" aria-label="Abrir menu">
        <span></span>
      </label>

      <nav class="nav" aria-label="Navegação principal">
        <a href="#tipografia">Tipografia</a>
        <a href="#imagens">Imagens</a>
        <a href="#cards">Cards</a>
      </nav>
    </div>
  </header>

  <main class="wrap">
    <section class="hero" id="tipografia">
      <h1>Testes de acessibilidade, imagens e responsividade</h1>
      <p>
        Aqui você valida foco visível, leitura com rem/em, estabilidade de imagem cor
        e diferenças entre auto-fit e auto-fill em cards.
      </p>
    </section>

    <section class="grid two">
      <article class="card good">
        <span class="tag">WCAG OK</span>

```



```

<h2>Como testar o menu com teclado</h2>
<p class="spaced">
  Pressione Tab até o botão do menu, observe o contorno de foco e use Espaço
  Tab deve seguir para os links do menu sem perder o destaque visual.
</p>
<p class="focus-tip">
  Regras: foco visível, operação por teclado e ordem lógica de navegação [web:55][web:84].
</p>
</article>

<article class="card bad">
  <span class="tag">ANTI-PADRÃO</span>
  <h2>Erro comum em menu</h2>
  <p class="spaced">
    Remover outline ou esconder o foco torna o menu difícil de usar para teclado
  </p>
  <p class="focus-tip">
    Nunca dependa só de hover para interação acessível [web:55][web:84].
  </p>
</article>
</section>

<section class="grid two" style="margin-top:1rem;">
  <article class="card good">
    <span class="tag">REM + EM</span>
    <h3>Escala previsível</h3>
    <div class="parent-rem">
      <p class="child-em">Este bloco usa 1.5em e reage ao tamanho do pai.</p>
      <p class="child-rem">Este bloco usa 1.5rem e mantém referência na raiz.</p>
    </div>
  </article>

  <article class="card bad">
    <span class="tag">RISCO</span>
    <h3>Multiplicação indesejada</h3>
    <div style="font-size: 24px;">
      <p style="font-size: 2em;">Se o pai mudar, o tamanho pode multiplicar.</p>
    </div>
  </article>
</section>

<section class="grid two" style="margin-top:1rem;">
  <article class="card good" id="imagens">
    <span class="tag">IMAGEM COM BREAKPOINTS</span>
    <h3>Picture + srcset</h3>
    <picture>
      <source
        media="(max-width: 768px)"
        srcset="https://images.unsplash.com/photo-1516321497487-e288fb19713f?auto=compress,format"
      />
      <source
        media="(max-width: 1200px)"
        srcset="https://images.unsplash.com/photo-1516321318423-f06f85e504b3?auto=compress,format"
      />
    </picture>
    
</picture>
<p class="focus-tip">O navegador escolhe a melhor fonte de imagem para cada c
</article>

<article class="card bad">
    <span class="tag">CLS / DEFORMAÇÃO</span>
    <h3>Imagem sem reserva explícita</h3>
    
    <p class="focus-tip">Sem proporção previsível, o layout pode deslocar quando
</article>
</section>

<section class="grid two" style="margin-top:1rem;">
    <article class="card good" id="cards">
        <span class="tag">AUTO-FIT</span>
        <h3>Cards que expandem</h3>
        <div class="grid cards-fit">
            <div class="sample">Card A</div>
            <div class="sample">Card B</div>
            <div class="sample">Card C</div>
        </div>
        <p class="focus-tip">auto-fit colapsa trilhas vazias e deixa os cards ocupando
</article>

    <article class="card good">
        <span class="tag">AUTO-FILL</span>
        <h3>Cards que preservam slots</h3>
        <div class="grid cards-fill">
            <div class="sample">Card A</div>
            <div class="sample">Card B</div>
            <div class="sample">Card C</div>
        </div>
        <p class="focus-tip">auto-fill mantém as colunas reservadas, mesmo quando há
</article>
</section>

<section class="grid two" style="margin:1rem 0 3rem;">
    <article class="card good">
        <span class="tag">VALIDAÇÃO WCAG</span>
        <h3>Checklist prático</h3>
        <ul>
            <li>Tab acessa menu e links.</li>
            <li>:focus-visible aparece com contraste alto.</li>
            <li>Texto continua legível com zoom.</li>
            <li>Imagens reservam espaço antes do carregamento.</li>
        </ul>
    </article>

```

```

<article class="card bad">
  <span class="tag">ERROS A EVITAR</span>
  <h3>Red flags</h3>
  <ul>
    <li>Outline removido.</li>
    <li>Uso exclusivo de hover.</li>
    <li>Larguras fixas grandes.</li>
    <li>Imagens sem width/height ou aspect-ratio.</li>
  </ul>
</article>
</section>
</main>
</body>
</html>

```

Regras WCAG aplicadas

O menu está preparado para uso por teclado e exibe foco visível com `:focus-visible`, o que atende a navegabilidade por teclado e visibilidade do foco.^{[91] [92] [90]}

A imagem principal usa `width/height` e `<picture>` com fontes alternativas, o que ajuda a evitar deslocamento de layout e melhora a adaptação a diferentes contextos de tela.^{[93] [94] [95]}

Os cards usam `auto-fit` e `auto-fill` para mostrar diferenças reais de comportamento sem comprometer legibilidade.^{[96] [97] [98]}

Como testar o menu

Use esta sequência no navegador: Tab até o botão, observe o contorno, pressione Espaço para abrir e continue com Tab para navegar pelos links.^{[99] [91]}

Se quiser simular auditoria, tente navegar sem mouse, com zoom aumentado e com foco visível mantido em todas as etapas.^{[92] [91]}

O que observar

- O foco precisa ser sempre perceptível, não só no hover.^{[91] [92]}
- A ordem de tabulação precisa ser lógica.^{[99] [91]}
- O menu não deve “sumir” sem um caminho de teclado para fechá-lo.^[91]
- As imagens precisam carregar sem saltar o layout.^{[95] [100]}

Posso montar a próxima versão com um **painel de testes ainda mais completo**, incluindo `line-height`, `letter-spacing`, `word-spacing` e um comparativo visual de zoom 100% vs 200%.

✱✱

1. <https://pt.linkedin.com/pulse/responsividade-web-técnicas-modernas-com-css3-e-html5-denis-manfroi-r9jge>
2. <https://kinsta.com/pt/blog/design-responsivo-web/>
3. <https://github.com/caelum/apostila-html-css-javascript/blob/master/14CA-responsividade-com-viewport-e-unidades-relativas.md>
4. <https://www.devfe.com.br/blog/as-melhores-praticas-de-design-responsivo-em-2024>

5. <https://yurideveloper.com.br/blog/index.php/2025/07/23/implementando-o-design-responsivo-com-css-grid-e-flexbox-3/>
6. <https://minacriativa.com.br/melhores-praticas-para-sites-responsivos/>
7. <https://diegolusa.github.io/web/css3/responsividade/>
8. <https://www.freecodecamp.org/espanol/news/aprende-unidades-de-medida-css-em-rem-vh-y-vw-con-ejemplos-de-codigos/>
9. <https://www.saurabhmisra.dev/responsive-images-html/>
10. <https://igormarcondes.com.br/como-criar-textos-responsivos-com-css>
11. https://developer.mozilla.org/pt-BR/docs/Web/HTML/Guides/Responsive_images
12. <https://malamute.digital/design-responsivo/>
13. <https://kinsta.com/pt/blog/melhores-praticas-web-design/>
14. <https://www.devmedia.com.br/responsive-design-dicas-para-tornar-seu-site-acessivel-em-qualquer-resolucao/28316>
15. <https://labex.io/pt/tutorials/css-responsive-web-design-for-all-screens-300113>
16. <https://www.instagram.com/arara.resultados/>
17. <https://www.sitesrecife.com/blog/tamanhos-de-fonte-e-responsividade-como-garantir-uma-bom-leitura-em-todos-os-dispositivos>
18. <https://www.alsacreations.com/article/lire/1621-responsive-images>
19. <https://github.com/topics/mobile-first-design>
20. <https://ipixel.com.sg/web-development/flexbox-vs-css-grid-2025-complete-guide-to-modern-css-layouts/>
21. <https://www.scribd.com/document/738039378/Las-principales-diferencias-entre-Flexbox-y-CSS-Grid-freeCodeCamp-org>
22. <https://elementor.com/blog/it/rem-vs-em-in-css-guida-dalla-a-alla-z/>
23. <https://www.alura.com.br/artigos/guia-de-unidades-no-css>
24. <https://www.dreamhost.com/blog/es/rem-vs-em-unidad-css/>
25. <https://full.services/glossario/breakpoints-responsivos/>
26. <https://www.youtube.com/watch?v=acJ0ZXhOrtM>
27. <https://fluxando.com.br/blog/otimizacao-imagens-site.html>
28. <https://www.saurabhmisra.dev/responsive-images-html/>
29. https://developer.mozilla.org/pt-BR/docs/Web/HTML/Guides/Responsive_images
30. <https://www.crazystack.com.br/2025-3/ultimate-guide-to-image-optimi>
31. <https://orquitol.com/pt/blog/otimizacao-de-imagens-responsivas-web-performance-core-web-v>
32. <https://pt.stackoverflow.com/questions/321861/banner-responsivo-esta-com-zoom>
33. <https://forum.bubble.io/t/css-object-fit-cover-in-the-new-responsive-beta/181705>
34. <https://pt.stackoverflow.com/questions/292454/resolucao-tamanho-da-imagem-em-mobile-css-aumentar-height>
35. <https://www.freecodecamp.org/portuguese/news/tutorial-de-imagens-responsivas-em-css-como-tornar-as-imagens-responsivas-com-o-css/>
36. <https://community.shopify.com/t/sobre-banner-desktop-e-mobile/205839>
37. <https://www.youtube.com/watch?v=2vCIGuFrh78>

38. <https://blog.hubspot.com/website/css-grid-vs-flexbox>
39. https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Core/CSS_layout/Responsive_Design
40. <https://www.freecodecamp.org/portuguese/news/design-responsivo-para-a-web-como-adaptar-um-site-a-celulares-e-tablets/>
41. <https://www.digitalocean.com/community/tutorials/css-cropping-images-object-fit>
42. <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/clamp>
43. <https://www.smashingmagazine.com/2022/01/modern-fluid-typography-css-clamp/>
44. <https://medium.com/@nunopereira.dev/css-grid-auto-fit-auto-fill-1cddcadbb4f3>
45. <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/repeat>
46. <https://dev.to/cwrcode/create-responsive-hamburger-menu-using-pure-css-only-2i83>
47. <https://www.youtube.com/watch?v=ub5ngVE4yq0>
48. <https://medium.com/@mateus2050/hamburguer-menu-html-and-css-only-c06364fa9bfd>
49. <https://forum.bubble.io/t/css-object-fit-cover-in-the-new-responsive-beta/181705>
50. <https://fluxando.com.br/blog/otimizacao-imagens-site.html>
51. https://www.linkedin.com/posts/bushra-al-za'reer-4a691812a_clamp-css-cascading-style-sheets-mdn-activity-7269362535810707456-c3fr
52. <https://www.youtube.com/watch?v=qll5BueXWD4>
53. <https://www.youtube.com/watch?v=R1ehrIk0oUU>
54. <https://github.com/mdn/content/blob/main/files/en-us/web/css/reference/values/clamp/index.md?plain=1>
55. <https://takazudomodular.com/pj/zcss/docs/layout/sizing/clamp-for-sizing/>
56. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Images_media_forms
57. <https://mdn.github.io/css-examples/object-fit-basics/>
58. https://www.w3schools.com/css/css3_object-fit.asp
59. https://developer.mozilla.org/en-US/docs/Web/Accessibility/Guides/Understanding_WCAG/Keyboard
60. <https://marcus.io/blog/accessible-overflow>
61. <https://developer.mozilla.org/en-US/docs/Web/CSS/overflow?v=control&rut=95588cc4d469e793f5068d048f7c3d6eb8967ed3cb7cb0dfe65080778e3351a7>
62. <https://developer.mozilla.org/de/docs/Web/CSS/Reference/Values/clamp>
63. <https://developer.mozilla.org/ja/docs/Web/CSS/Reference/Values/clamp>
64. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Box_sizing/Aspect_ratios
65. <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/font-size>
66. <https://stackoverflow.com/questions/75494407/layout-shifts-and-width-height-attributes-on-img>
67. <https://github.com/jakearchibald/jakearchibald.com/blob/main/static-build/posts/2022/07/img-aspect-ratio/index.md>
68. <https://refine.dev/blog/rem-vs-em/>
69. <https://element84.com/software-engineering/web-development/responsive-typography/>
70. <https://www.linkedin.com/pulse/designers-guide-accessible-typography-ferg-flannery-u5dbf>
71. <https://www.dennisdeacon.com/accessibility/testing-methods-text-spacing/>

72. <https://www.youtube.com/watch?v=oYkATmECg3M>
73. <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/clamp>
74. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Images_media_forms
75. https://developer.mozilla.org/en-US/docs/Web/Accessibility/Guides/Understanding_WCAG/Keyboard
76. <https://stackoverflow.com/questions/78312465/how-to-fix-layout-shift-for-responsive-images-that-use-the-picture-source-html>
77. <https://www.w3.org/WAI/WCAG22/Techniques/css/C21>
78. <https://refine.dev/blog/rem-vs-em/>
79. <https://www.w3.org/WAI/WCAG21/Understanding/text-spacing>
80. <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/font-size>
81. <https://remtopx.com/blog/responsive-typography-best-practices/>
82. <https://medium.com/@JanKoritak/how-to-prevent-cumulative-layout-shift-for-responsive-images-571ccc7316f8>
83. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Box_sizing/Aspect_ratios
84. <https://stackoverflow.com/questions/75494407/layout-shifts-and-width-height-attributes-on-img>
85. <https://github.com/web-accessibility-io/web-accessibility.io/blob/main/data/playbook/1-4-12-text-spacing.d.mdx>
86. <https://www.accessitree.com/wcag-ultimate-guide/ensure-content-adapts-to-user-adjusted-text-spacing/>
87. <https://stackoverflow.com/questions/70784566/can-i-fix-cumulative-layout-shift-without-knowing-the-width-and-height-of-an-image>
88. https://dev.to/bnevilleoneill/the-elements-of-responsive-typography-fok?comments_sort=latest
89. <https://dev.to/dainemawer/best-cls-practices-for-images-4913>
90. https://mdn.org/en-US/docs/Web/Accessibility/Understanding_WCAG/Keyboard
91. https://developer.mozilla.org/en-US/docs/Web/Accessibility/Guides/Understanding_WCAG/Keyboard
92. <https://getwcag.com/nl/accessibility-guide/focus-visible>
93. <https://developer.mozilla.org/en-US/docs/Web/API/HTMLImageElement/srcset>
94. <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/picture>
95. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Box_sizing/Aspect_ratios
96. <https://medium.com/@mandy.michael/you-dont-need-a-media-query-for-that-2-responsive-columns-2582c6ea8700>
97. <https://shunnien.github.io/2018/03/24/css-grid-12/>
98. <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/grid-template-columns>
99. https://developer.mozilla.org/ja/docs/Web/Accessibility/Guides/Understanding_WCAG/Keyboard
100. <https://stackoverflow.com/questions/75494407/layout-shifts-and-width-height-attributes-on-img>
101. https://blog.csdn.net/gitblog_01146/article/details/154416757